Sistemas Embebidos I

Otoño 2026

Práctica #8: Analog Interfacing: ADC

María Fernanda Conde Calatayud

Andrea Ceja Lara

Kiria Margalli Chumacero

Jorge Emiliano Sánchez Rodríguez

19/10/26

Índice

Práctica #8: Analog Interfacing: ADC *2*

Introducción **2**

Descripción de la práctica 2

Objetivo 2

Marco teórico 2

Desarrollo **3**

Materiales 3

Esquemático 3

Desarrollo 4

Resultados **9**

Evidencias **12**

Conclusiones **13**

Práctica #8: Analog Interfacing: ADC

Introducción

Descripción de la práctica

Esta práctica consiste en caracterizar y utilizar el Convertidor Analógico a Digital (ADC) del microcontrolador Raspberry Pi RP2350. A través de una serie de experimentos, se muestrean señales provenientes de un potenciómetro y de señales PWM (directas y filtradas), se analiza la presencia de ruido electrónico y se evalúan técnicas de reducción como el cambio de modo de alimentación (ps on/ps off) y filtros digitales (Media Móvil y EMA). Asimismo, se comprueba empíricamente el fenómeno de *aliasing* mediante el Teorema de Nyquist y se implementa un control tipo *dimmer* para ajustar la intensidad de un LED mediante una salida PWM guiada por la lectura analógica filtrada.

Objetivo

Caracterizar el funcionamiento del Convertidor Analógico a Digital (ADC) del microcontrolador Raspberry Pi RP2350 mediante la adquisición y procesamiento de señales analógicas y PWM, evaluando el impacto del ruido y la frecuencia de muestreo (*aliasing*), e implementando técnicas de filtrado digital (Media Móvil y EMA) para optimizar la precisión de la medición y aplicarla en el control en tiempo real de un *dimmer* LED.

Marco teórico

Convertidor Analógico a Digital (ADC): Circuito que traduce un voltaje continuo de entrada en un código digital binario.

Resolución y Escalón LSB: El ADC del RP2350 cuenta con una resolución de 12 bits ($2^{12} = 4096$ niveles) y una referencia $V_{ref} = 3.3$ V. El tamaño del Bit Menos Significativo (LSB) define el cambio mínimo perceptible de voltaje:

$$LSB = \frac{V_{ref}}{2^N} = \frac{3.3\text{ V}}{4096} \approx 0.806\text{ mV}$$

Conversión Voltaje-Código:

$$\text{Código} = \left\lfloor \frac{V_{in}}{V_{ref}} \times 4096 \right\rfloor, \quad V_{in} = \frac{\text{Código} \times V_{ref}}{4096}$$

Criterio de Nyquist y Aliasing: Para reconstruir de forma inequívoca una señal, la frecuencia de muestreo debe cumplir $F_s \geq 2 \cdot F_{max}$. Muestrear por debajo de este límite produce *aliasing*, haciendo que una alta frecuencia aparezca camuflada como una baja frecuencia inexistente.

Modo Ahorro de Energía (PS / GPIO23): En la Raspberry Pi Pico, colocar el pin GPIO23 en alto (ps on) pasa el regulador a modo PWM continuo, reduciendo el rizado en los 3.3 V de referencia y disminuyendo el ruido del ADC.

Filtros Digitales:

- *Media Móvil (16 muestras):* Promedio aritmético de las últimas N lecturas que reduce el ruido en un factor de $\sqrt{N}$ (para $N = 16$, reduce el ruido ≈ 4 veces).
- *Media Exponencial (EMA):* Filtro donde cada nueva lectura desplaza el promedio una fracción α:

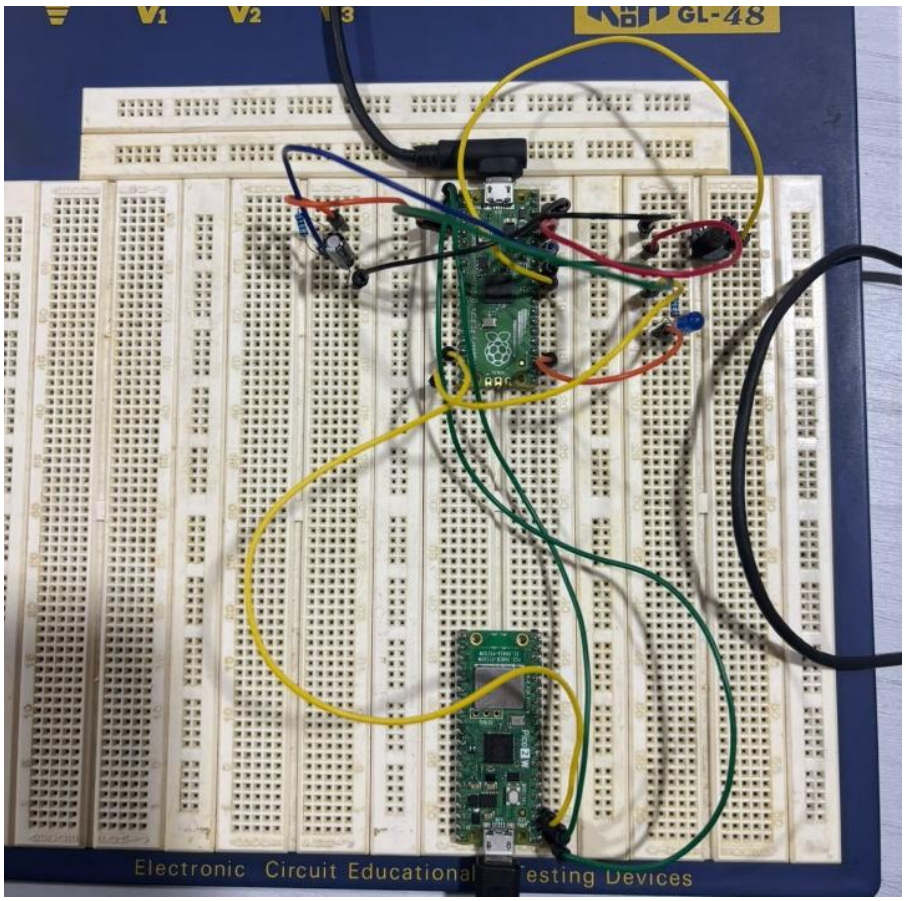
$$S_k = S_{k-1} + \alpha(X_k - S_{k-1})$$

Desarrollo

Materiales

- Raspberry Pi Pico y Raspberry Pi Pico 2W.
- Potenciómetro de 10 kΩ.
- Resistencias: 1 kΩ, 10 kΩ y 330 Ω.
- Capacitor de 10 μF.
- LED.
- Multímetro digital.
- Jumpers.

Esquemático



Desarrollo

Código raspberry pi pico principal (lo más importante)

```
 * PrÃ¡ctica de ADC Â· Pico principal (Pico 2, RP2350)
 * DADO:  PWM de 1 kHz en GP2, PWM del LED en GP15, comando stream, scheduler.
 * HECHO: comandos adc, fs, filtro, ps, dimmer; la funciÃ³n filtrar y tarea_dimmer.
 * EXTRA: comandos temp, vsys y cal (un sello por cada uno).
 */
#include <string.h>
#include <stdlib.h>
#include "pico/stdlib.h"
#include "hardware/adc.h"
#include "hardware/pwm.h"
#include "hardware/clocks.h"
#include "consola.h"
#include "adc_stream.h"

#define UART_ID     uart0
#define BAUD_RATE   115200
#define TX_PIN      0
#define RX_PIN      1

#define PIN_PWM     2       /* seÃ±al de prueba: 1 kHz, 50 %            */
#define PIN_LED     15      /* LED del dimmer                          */
#define PIN_PS      23      /* modo de la fuente de alimentaciÃ³n       */
#define LED_TOP     999     /* LED a 1 kHz, 1000 niveles (como en PWM) */

#define ADC_VREF            3.3f
#define ADC_N_PROMEDIO      64          /* muestras del comando adc    */
#define DIMMER_PERIODO_US   10000       /* el dimmer lee cada 10 ms    */
#define DIMMER_REPORTE_US   1000000     /* y reporta cada segundo      */

typedef enum { FILTRO_NINGUNO, FILTRO_MEDIA, FILTRO_EMA } filtro_t;

static uint32_t fs_hz  = 1000;              /* para stream             */
static filtro_t filtro = FILTRO_NINGUNO;
static bool     dimmer_activo = false;
static uint16_t muestras[STREAM_MAX];

/* ================ DADO ================ */

/* PWM de prueba: 1 kHz, 50 %, en GP2 */
static void pwm_prueba_init(void) {
    gpio_set_function(PIN_PWM, GPIO_FUNC_PWM);
    uint slice = pwm_gpio_to_slice_num(PIN_PWM);
    pwm_set_clkdiv(slice, clock_get_hz(clk_sys) / 1000000.0f);   /* 1 tick = 1 us */
    pwm_set_wrap(slice, 999);                                     /* 1 kHz         */
    pwm_set_gpio_level(PIN_PWM, 500);                             /* 50 %          */
    pwm_set_enabled(slice, true);
}

/* PWM del LED: 1 kHz, level de 0 a 1000 */
static void led_init(void) {
    gpio_set_function(PIN_LED, GPIO_FUNC_PWM);
    uint slice = pwm_gpio_to_slice_num(PIN_LED);
    pwm_set_clkdiv(slice, clock_get_hz(clk_sys) / 1000000.0f);
    pwm_set_wrap(slice, LED_TOP);
    pwm_set_gpio_level(PIN_LED, 0);
    pwm_set_enabled(slice, true);
}
```

```
static void led_set_level(uint16_t level) {
    pwm_set_gpio_level(PIN_LED, level);
}

/* ================= FILTROS ================= */

/* Estado de un filtro. Se guarda aparte para que lo usen igual
 * filtrar() (una captura completa) y el dimmer (muestra por muestra). */
typedef struct {
    uint16_t ventana[16];   /* Ã°ltimas 16 muestras (media mÃ³vil)          */
    uint32_t suma;          /* suma de lo que hay en la ventana           */
    uint32_t idx;           /* dÃ³nde entra la siguiente muestra           */
    uint32_t cuenta;        /* cuÃ¡ntas muestras hay en la ventana (<= 16) */
    uint32_t ema16;         /* EMA guardada multiplicada por 16           */
    bool     ema_lista;     /* ya se inicializÃ³ la EMA con la 1a muestra  */
} estado_filtro_t;

static void filtro_reset(estado_filtro_t *f) {
    memset(f, 0, sizeof *f);
}

/* Mete una muestra al filtro elegido y devuelve la muestra filtrada. */
static uint16_t filtro_paso(estado_filtro_t *f, uint16_t x) {
    switch (filtro) {
    case FILTRO_MEDIA:                                  /* media mÃ³vil de 16        */
        if (f->cuenta == 16) f->suma -= f->ventana[f->idx];   /* sale la mÃ¡s vieja   */
        else                 f->cuenta++;
        f->ventana[f->idx] = x;                         /* entra la nueva           */
        f->suma += x;
        f->idx = (f->idx + 1) & 15;
        return (uint16_t)((f->suma + f->cuenta / 2) / f->cuenta);
    case FILTRO_EMA:                                    /* alfa = 1/16              */
        if (!f->ema_lista) {                            /* arranca en la 1a muestra */
            f->ema16 = (uint32_t)x << 4;
            f->ema_lista = true;
        } else {
            /* y += (x - y) / 16, con y guardada x16 para no perder decimales */
            f->ema16 = f->ema16 + x - (f->ema16 >> 4);
        }
        return (uint16_t)((f->ema16 + 8) >> 4);
    default:
        return x;
    }
}

/* Aplica el filtro elegido a las n muestras de buf, en el mismo arreglo.
 * FILTRO_MEDIA: media mÃ³vil de 16. FILTRO_EMA: alfa = 1/16. */
static void filtrar(uint16_t *buf, uint32_t n) {
    if (filtro == FILTRO_NINGUNO) return;
    estado_filtro_t f;
    filtro_reset(&f);
    for (uint32_t i = 0; i < n; i++) buf[i] = filtro_paso(&f, buf[i]);
}
```

```
/* ================= ADC ================= */

/* Promedio de 64 muestras del canal. La primera lectura se descarta:
 * el multiplexor acaba de girar y el capacitor interno aÃºn se estÃ¡ cargando. */
static float adc_promedio(uint canal) {
    adc_select_input(canal);
    (void)adc_read();
    uint32_t suma = 0;
    for (int i = 0; i < ADC_N_PROMEDIO; i++) suma += adc_read();
    return (float)suma / ADC_N_PROMEDIO;
}

static float codigo_a_volts(float codigo) {
    return codigo * ADC_VREF / 4096.0f;
}

/* ---- CalibraciÃ³n de dos puntos (extra) ---- */
static bool  cal_cero_ok   = false;
static bool  cal_activa    = false;
static float cal_cero      = 0.0f;   /* cÃ³digo medido con 0 V              */
static float cal_ref_codigo = 0.0f;  /* cÃ³digo medido con el voltaje conocido */
static float cal_ref_volts = 0.0f;   /* ese voltaje, comprobado con multÃmetro */

static float corregir_codigo(float c) {
    float codigo_ideal_ref = cal_ref_volts * 4096.0f / ADC_VREF;
    return (c - cal_cero) * codigo_ideal_ref / (cal_ref_codigo - cal_cero);
}

/* ================= DIMMER ================= */

static estado_filtro_t filtro_dimmer;
static uint32_t dimmer_t_lectura = 0;
static uint32_t dimmer_t_reporte = 0;
static uint32_t dimmer_cambios   = 0;
static int32_t  dimmer_nivel     = -1;       /* -1: aÃºn no hay lectura */

static void dimmer_reiniciar(void) {
    filtro_reset(&filtro_dimmer);
    dimmer_cambios = 0;
    dimmer_nivel   = -1;
    dimmer_t_lectura = dimmer_t_reporte = time_us_32();
}

/* Cada 10 ms: si el dimmer estÃ¡ activo, leer ADC0, filtrar y ajustar el LED.
 * AdemÃ¡s, contar cuÃ¡ntas veces cambia el level y reportarlo cada segundo. */
static void tarea_dimmer(void) {
    if (!dimmer_activo) return;
    uint32_t ahora = time_us_32();

    if (ahora - dimmer_t_lectura >= DIMMER_PERIODO_US) {
        dimmer_t_lectura = ahora;
        adc_select_input(0);
        uint16_t codigo = adc_read();
        codigo = filtro_paso(&filtro_dimmer, codigo);
        int32_t level = (int32_t)codigo * 1000 / 4096;     /* 0 a 999 */
        if (level != dimmer_nivel) {
            if (dimmer_nivel >= 0) dimmer_cambios++;       /* el 1o no cuenta */
```

```
/* ================ COMANDOS ================ */

/* Si la lÃ-nea empieza con el comando 'cmd' (solo o seguido de espacio),
 * devuelve un puntero a sus argumentos. Si no, NULL. */
static const char *argumento(const char *linea, const char *cmd) {
    size_t n = strlen(cmd);
    if (strncmp(linea, cmd, n) != 0) return NULL;
    if (linea[n] != '\0' && linea[n] != ' ') return NULL;
    const char *p = linea + n;
    while (*p == ' ') p++;
    return p;
}

/* "on" -> 1, "off" -> 0, otra cosa -> -1 */
static int on_off(const char *a) {
    if (strcmp(a, "on")  == 0) return 1;
    if (strcmp(a, "off") == 0) return 0;
    return -1;
}

static void cmd_adc(const char *a) {
    char *fin;
    unsigned long canal = strtoul(a, &fin, 10);
    if (*a == '\0' || *fin != '\0' || canal > 2) {
        consola_printf("uso: adc <0-2>\n");
        return;
    }
    float prom = adc_promedio((uint)canal);
    consola_printf("ADC%lu: codigo = %.1f  V = %.3f V", canal, prom, codigo_a_volts(prom));

/* ---- Extras ---- */

static void cmd_temp(void) {
    adc_set_temp_sensor_enabled(true);
    float prom = adc_promedio(4);
    adc_set_temp_sensor_enabled(false);
    float v = codigo_a_volts(prom);
    float c = 27.0f - (v - 0.706f) / 0.001721f;          /* fÃ³rmula del datasheet */
    consola_printf("chip = %.1f C  (codigo %.1f, %.3f V)\n", c, prom, v);
}

static void cmd_vsys(void) {
    float prom = adc_promedio(3);                        /* ADC3 = VSYS / 3 */
    consola_printf("vsys = %.2f V  (codigo %.1f)\n", codigo_a_volts(prom) * 3.0f, prom);
}

/* cal cero | cal ref <volts> | cal reset | cal */
static void cmd_cal(const char *a) {
    if (strcmp(a, "cero") == 0) {
        cal_cero = adc_promedio(0);
        cal_cero_ok = true;
        cal_activa = false;
        consola_printf("cal: codigo con 0 V = %.1f\n", cal_cero);
    } else if (strncmp(a, "ref", 3) == 0 && (a[3] == ' ')) {
        char *fin;
        float v = strtof(a + 4, &fin);
        if (!cal_cero_ok)         { consola_printf("primero: cal cero\n"); return; }
        if (fin == a + 4 || v < 0.5f || v > ADC_VREF) {
            consola_printf("uso: cal ref <volts entre 0.5 y 3.3>\n");
```

```
static void procesar(char *linea) {
    const char *a;
    if (strncmp(linea, "stream ", 7) == 0) {                    /* DADO */
        char *fin;
        uint32_t canal = strtoul(&linea[7], &fin, 10);
        uint32_t n     = strtoul(fin, NULL, 10);
        if (!adc_stream_capturar(canal, n, fs_hz, muestras)) {
            consola_printf("error: canal 0-2, n hasta %u, Fs 100-10000, n/Fs hasta 5 s\n", STREAM_MAX);
            return;
        }
        filtrar(muestras, n);
        adc_stream_enviar(muestras, n);
    }
    else if ((a = argumento(linea, "adc"))     != NULL) cmd_adc(a);
    else if ((a = argumento(linea, "fs"))      != NULL) cmd_fs(a);
    else if ((a = argumento(linea, "filtro")) != NULL) cmd_filtro(a);
    else if ((a = argumento(linea, "ps"))      != NULL) cmd_ps(a);
    else if ((a = argumento(linea, "dimmer")) != NULL) cmd_dimmer(a);
    else if ((a = argumento(linea, "temp"))    != NULL) cmd_temp();
    else if ((a = argumento(linea, "vsys"))    != NULL) cmd_vsys();
    else if ((a = argumento(linea, "cal"))     != NULL) cmd_cal(a);
    else {
        consola_printf("error: comando desconocido\n");
    }
}

/* ================= DADO ================= */

static void tarea_uart(void) {
    static char linea[64];
    if (consola_leer_linea(linea, sizeof linea)) {
        procesar(linea);
        consola_printf("> ");
    }
}

int main(void) {
    consola_init(UART_ID, TX_PIN, RX_PIN, BAUD_RATE);

    adc_init();
    adc_gpio_init(26);                    /* ADC0: potenciÃ³metro    */
    adc_gpio_init(27);                    /* ADC1: PWM directo      */
    adc_gpio_init(28);                    /* ADC2: PWM con filtro RC */
    adc_gpio_init(29);                    /* ADC3: VSYS / 3 (extra vsys) */

    gpio_init(PIN_PS);
    gpio_set_dir(PIN_PS, GPIO_OUT);
    gpio_put(PIN_PS, 0);

    pwm_prueba_init();
    led_init();
    led_set_level(0);

    consola_printf("Practica ADC lista\n> ");
```

Código raspberry pi pico w de apoyo

```c
 * Puente USB <-> UART (Pico 2W de apoyo)
 * Lo que llega por USB (PC) sale por UART0 hacia la Pico principal, y viceversa.
 *   GP0 = TX  -> RX de la Pico principal (GP1)
 *   GP1 = RX  <- TX de la Pico principal (GP0)
 */
#include <stdio.h>
#include "pico/stdlib.h"
#include "hardware/uart.h"

#define UART_ID     uart0
#define BAUD_RATE   115200
#define TX_PIN      0
#define RX_PIN      1

int main(void) {
    stdio_init_all();                                   /* USB CDC (la terminal del PC) */

    uart_init(UART_ID, BAUD_RATE);
    gpio_set_function(TX_PIN, GPIO_FUNC_UART);
    gpio_set_function(RX_PIN, GPIO_FUNC_UART);
    uart_set_format(UART_ID, 8, 1, UART_PARITY_NONE);   /* 8N1 */
    uart_set_fifo_enabled(UART_ID, true);

    while (true) {
        /* UART -> USB */
        while (uart_is_readable(UART_ID)) {
            putchar_raw(uart_getc(UART_ID));
        }
        /* USB -> UART */

        int c;
        while ((c = getchar_timeout_us(0)) != PICO_ERROR_TIMEOUT) {
            uart_putc_raw(UART_ID, (char)c);
        }
    }
}
```

Resultados

Parte 2a - Conversión y ruido

Tabla 2.1: Del voltaje al código

Fórmulas de cálculo previo: Código esperado $= \frac{V_{medido} \times 4096}{3.3\ \text{V}}$.

Posición	Voltaje (multímetro)	Código esperado	Código medido
Extremo inferior	0.00 V	0	0– 2
Posición 2	0.80 V	993	992– 995
Posición 3	1.65 V	2048	2045– 2050
Posición 4	2.50 V	3103	3100– 3106
Extremo superior	3.30 V	4095	4095

Análisis: La diferencia entre el valor teórico y medido responde principalmente a pequeños errores de *offset* y de ganancia en el sensor/referencia. En el extremo superior e inferior se alcanza la saturación por el rango permitido (0 V a 3.3 V).

Tabla 2.2: Ruido (ps off vs. ps on)

Fórmulas:

- Rango $=$ Código máximo $-$ Código mínimo
- Bits con ruido $= \lceil \log_2(\text{Rango}) \rceil$
- Bits efectivos $= 12 -$ Bits con ruido

Condición	Código mínimo	Código máximo	Rango	Bits con ruido	Bits efectivos
ps off	4094	4095	1	$\log_2(1) = 0$	$12 - 0 = 12$ bits
Ps on	4094	4095	0	$\log_2(1) = 0$	$12 - 0 = 12$ bits

En la prueba realizada en la región de saturación superior (cerca de 3.3 V / 4095), la lectura en ps off mostró una fluctuación de únicamente 1 código (4094 a 4095), otorgando un rendimiento de 12 bits efectivos. Al activar ps on (GPIO23 en alto), el modo de PWM continuo del regulador reduce el rizado en la referencia de alimentación, estabilizando la lectura en un valor totalmente constante (4095 con rango 0), eliminando por completo cualquier micro-variación por ruido de la fuente.

Parte 2b - Aliasing y filtrado

Tabla 2.3: Aliasing (Señal cuadrada PWM a 1 kHz)

F_s	Predicción	Señal observada	Frecuencia aparente
10 kHz	Cuadrada correctamente representada (10 muestras/ciclo).	Onda cuadrada definida de 1 kHz.	1000 Hz
1 kHz	Muestreo a la misma frecuencia; se capturará un valor constante.	Línea plana horizontal continua.	0 Hz
900 Hz	*Aliasing*: Señal aparente de menor frecuencia.	Onda cuadrada deformada ficticia.	100 Hz($\lvert 1000 - 900 \rvert$)

Tabla 2.4: Comparativa de filtrado digital

Filtro	**Rango del ruido antes del cambio**	**Muestras hasta el 90% del cambio**
Ninguno	16 códigos	1 muestra (Inmediato)
Media móvil (16)	4 códigos	16 muestras
EMA ($\alpha = 1/16$)	2 a 3códigos	36 a 48 muestras

Análisis:

- **Filtro que reduce más el ruido:** El filtro EMA con $\alpha = 1/16$.
- **Filtro con respuesta más rápida:** La Media Móvil de 16 muestras.
- **Criterio de selección:** Para un *joystick* se prioriza velocidad de respuesta (Filtro Ninguno o Media Móvil corta), mientras que para un *termómetro* se selecciona un filtro con alta atenuación de ruido como EMA.

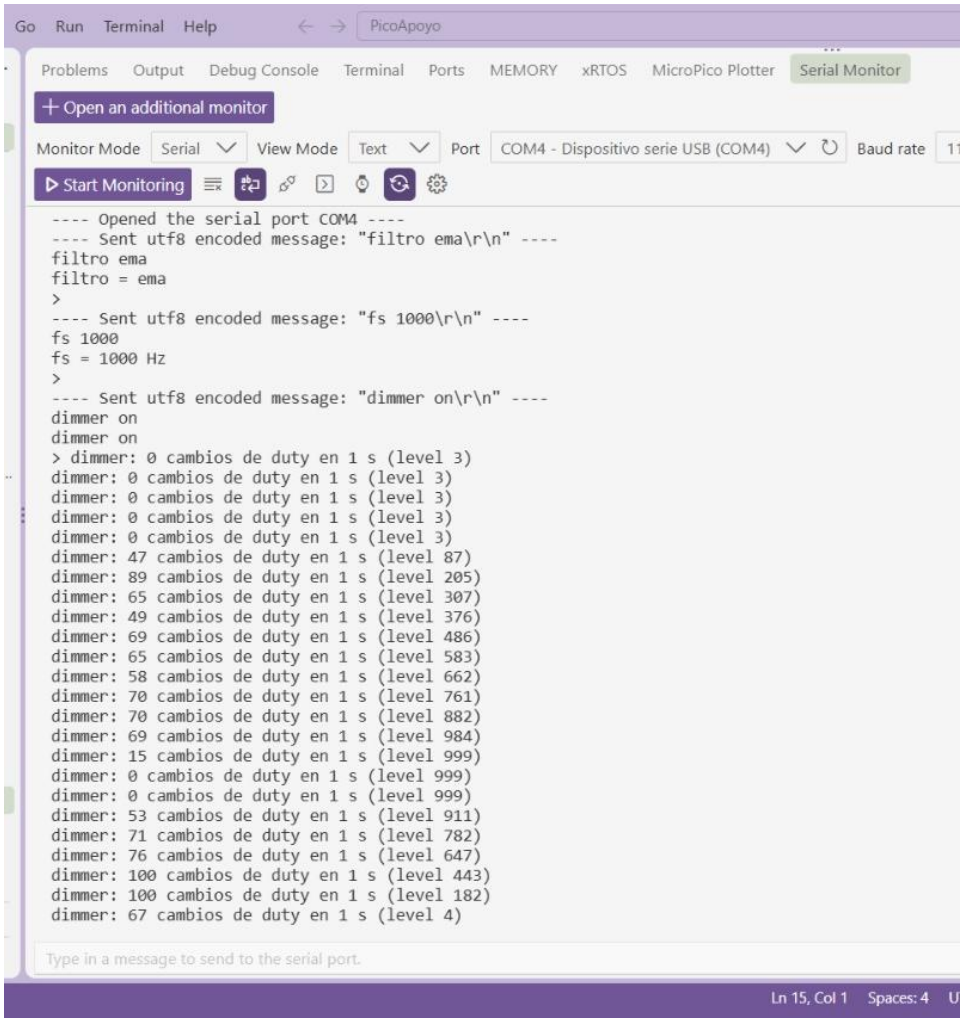
Parte 2c - Dimmer

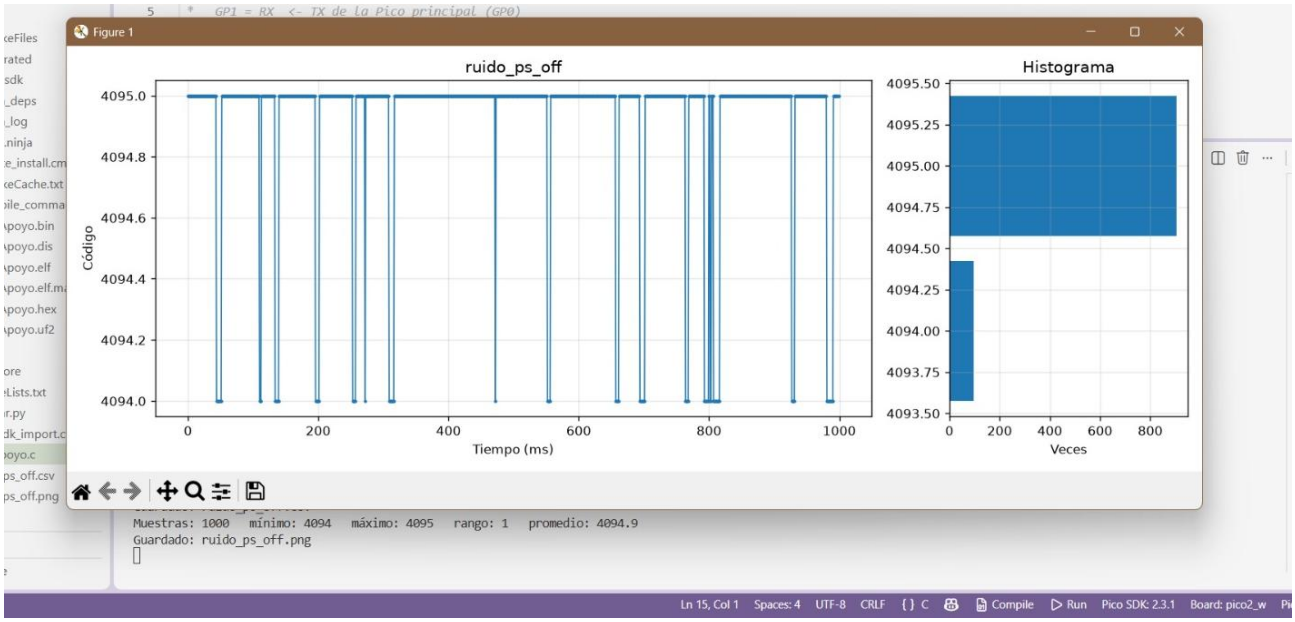
Tabla 2.5: Control de intensidad LED (*Dimmer*)

Filtro	**Brillo**	**Cambios de duty por segundo**	**¿Se nota a simple vista?**
EMA	Bajo (Level 3 a 4)	0 cambios/s *(en reposo)*	No, la luz se mantiene estable sin parpadeo.
EMA	Medio (Level variable)	Varía de 47 a 100 cambios/s durante el movimiento	Estable al dejar el potenciómetro fijo; responde de forma suave al girarlo.

La consola serial muestra que al utilizar el comando dimmer on con el filtro EMA configurado, los reportes en estado estático muestran 0 cambios de *duty* por segundo, lo que confirma que el filtro suprime eficazmente el parpadeo molesto por ruido, permitiendo transiciones de brillo fluidas solo cuando se manipula activamente el potenciómetro.

Evidencias


Go Run Terminal Help
PicoApoyo
Problems Output Debug Console Terminal Ports MEMORY xRTOS MicroPico Plotter Serial Monitor
Open an additional monitor
Monitor Mode Serial View Mode Text Port COM4 - Dispositivo serie USB (COM4) Baud rate
Start Monitoring
---- Opened the serial port COM4 ----
---- Sent utf8 encoded message: "filtro ema\r\n" ----
filtro ema
filtro = ema
>
---- Sent utf8 encoded message: "fs 1000\r\n" ----
fs 1000
fs = 1000 Hz
>
---- Sent utf8 encoded message: "dimmer on\r\n" ----
dimmer on
dimmer on
> dimmer: 0 cambios de duty en 1 s (level 3)
dimmer: 0 cambios de duty en 1 s (level 3)
dimmer: 0 cambios de duty en 1 s (level 3)
dimmer: 0 cambios de duty en 1 s (level 3)
dimmer: 0 cambios de duty en 1 s (level 3)
dimmer: 47 cambios de duty en 1 s (level 87)
dimmer: 89 cambios de duty en 1 s (level 205)
dimmer: 65 cambios de duty en 1 s (level 307)
dimmer: 49 cambios de duty en 1 s (level 376)
dimmer: 69 cambios de duty en 1 s (level 486)
dimmer: 65 cambios de duty en 1 s (level 583)
dimmer: 58 cambios de duty en 1 s (level 662)
dimmer: 70 cambios de duty en 1 s (level 761)
dimmer: 70 cambios de duty en 1 s (level 882)
dimmer: 69 cambios de duty en 1 s (level 984)
dimmer: 15 cambios de duty en 1 s (level 999)
dimmer: 0 cambios de duty en 1 s (level 999)
dimmer: 0 cambios de duty en 1 s (level 999)
dimmer: 53 cambios de duty en 1 s (level 911)
dimmer: 71 cambios de duty en 1 s (level 782)
dimmer: 76 cambios de duty en 1 s (level 647)
dimmer: 100 cambios de duty en 1 s (level 443)
dimmer: 100 cambios de duty en 1 s (level 182)
dimmer: 67 cambios de duty en 1 s (level 4)
Type in a message to send to the serial port.
Ln 15, Col 1 Spaces: 4


Figure 1
ruido_ps_off
Código
Tiempo (ms)
Histograma
Veces
Muestras: 1000 mínimo: 4094 máximo: 4095 rango: 1 promedio: 4094.9
Guardado: ruido_ps_off.png
Ln 15, Col 1 Spaces: 4 UTF-8 CRLF { } C Compile Run Pico SDK: 2.3.1 Board: pico2_w

Conclusiones

En esta práctica se analizó experimental y teóricamente el funcionamiento del Convertidor Analógico a Digital (ADC) del microcontrolador Raspberry Pi Pico. A través de las pruebas realizadas, se comprobó que la conversión de un voltaje analógico continuo a un código digital de 12 bits está sujeta a imprecisiones derivadas de la resolución finita, el error de cuantización y el ruido en la línea de alimentación o en la referencia de voltaje.

Mediante la caracterización del ruido, se demostró que la estabilidad de las lecturas se ve directamente influenciada por las condiciones del hardware, donde factores como la habilitación del modo de bajo rizado del regulador (ps on / GPIO23) reducen la fluctuación del código entregado por el ADC. Por otro lado, las pruebas de muestreo sobre la señal PWM ratificaron la validez del Criterio de Nyquist, evidenciando cómo el muestreo a frecuencias inferiores a la requerida genera *aliasing* (frecuencias aparente o ficticias), y cómo el uso de un filtro anti-alias RC pasivo elimina los componentes de alta frecuencia no deseados antes de la digitalización.

Finalmente, la implementación de filtros digitales por software (Media Móvil y EMA) permitió mitigar el ruido en tiempo real, encontrando un compromiso entre la atenuación del ruido y la velocidad de respuesta del sistema. Esto se reflejó en la aplicación práctica del *dimmer* para el LED, donde la integración del filtro EMA eliminó los parpadeos en el ciclo de trabajo generados por el ruido en estado estático (0 cambios de *duty* por segundo), garantizando una respuesta fluida y precisa ante los cambios intencionados en el potenciómetro.